

1. What are the differences between Topological Sort and DFS?

DFS	TOPOLOGICAL SORT
1. A graph traversal algorithm that explores as far as possible along each branch before backtracking.	1. A linear ordering of nodes in a DAG such that for every directed edge $u \rightarrow v$, node u comes before v .
2. Works on all graphs (directed, undirected, cyclic, acyclic).	2. Works only on Directed Acyclic Graphs (DAGs). If there's a cycle, topological sort is impossible.
3. Mainly used for traversal, cycle detection, connected components, path finding	3. Mainly used for dependency resolution (task scheduling, build systems, course prerequisites).
4. Produces a <i>traversal order</i> of vertices. • The order depends on the chosen starting node and the order of neighbors. • The result is not unique and does not guarantee respect for dependency constraints in the graph.	4. Produces a <i>valid linear ordering</i> of vertices in a DAG. • Ensures that for every directed edge $u \rightarrow v$, vertex u appears before vertex v. • Multiple valid orderings may exist, but all respect the graph's dependency rules.
5. Visits nodes recursively and marks them visited, then backtracks.	5. Similar to DFS but when a node finishes (no unvisited neighbors left), it gets pushed to a stack. The final ordering is the reverse of finishing times.
6. Can detect cycles if we track back edges.	6. Existence of a cycle means no valid topological ordering.

Pseudocode(DFS):

DFS(Graph G):

mark all vertices as unvisited

for each vertex v in G :

if v is unvisited:

DFS-Visit(v)

DFS-Visit(v):

mark v as visited

print(v) // or record v in traversal order

for each neighbor u of v :

if u is unvisited:

DFS-Visit(u)

Pseudocode (Topological sort using DFS):

Topological-Sort(Graph G):

mark all vertices as unvisited

create an empty stack S

for each vertex v in G :

if v is unvisited:

DFS-Topo(v , S)

return reverse(S) // or pop all from S

DFS-Topo(v , S):

mark v as visited

for each neighbor u of v :

if u is unvisited:

DFS-Topo(u , S)

push v onto S // add to stack when fully processed

2. Implement detection of Strongly Connected Components using DFS.

To detect Strongly Connected Components (SCCs) in a directed graph, we usually use Kosaraju's Algorithm (two-pass DFS).

Kosaraju's Algorithm (DFS-based) for SCCs (Idea):

1. Do a DFS on the original graph and record finishing times (store vertices in a stack after recursion ends).
2. Reverse the direction of all edges (get the transpose graph).
3. Do DFS again on the reversed graph, but in the order of vertices popped from the stack.
4. Each DFS tree gives one strongly connected component.

Kosaraju's Algorithm (DFS-based) for SCCs (Pseudocode):

Kosaraju-SCC(Graph G):

mark all vertices as unvisited

create an empty stack S

// Step 1: DFS to fill stack by finishing times

for each vertex v in G :

if v is unvisited:

DFS-Fill(v , S)

// Step 2: Reverse the graph

$G_T = \text{Transpose}(G)$

// Step 3: DFS on reversed graph in stack order

mark all vertices as unvisited

while S is not empty:

$v = \text{pop}(S)$

if v is unvisited:

currentSCC = []

DFS-Collect(v , G_T , currentSCC)

print("SCC:", currentSCC)

DFS-Fill(v , S):

mark v as visited

for each neighbor u of v :

if u is unvisited:

DFS-Fill(u , S)

push v onto S // store finishing order

```
DFS-Collect(v, G_T, component):  
    mark v as visited  
    add v to component  
    for each neighbor u of v in G_T:  
        if u is unvisited:  
            DFS-Collect(u, G_T, component)
```

3. Compare the DFS recursive approach and the iterative approach.

Recursive DFS Pseudocode:

```
DFS-Recursive(Graph G, v, visited):  
    mark v as visited  
    print(v)           // process node  
    for each neighbor u of v:  
        if u is not visited:  
            DFS-Recursive(G, u, visited)  
  
for each vertex v in G:  
    if v is not visited:  
        DFS-Recursive(G, v, visited)
```

Iterative DFS Pseudocode:

DFS-Iterative-Complete(Graph G):

mark all vertices as unvisited

for each vertex v in G:

if v is not visited:

DFS-Iterative(G, v)

DFS-Iterative(Graph G, start):

create empty stack S

push start onto S

mark start as visited

while S is not empty:

v = pop(S)

print(v)

neighbors = get_neighbors(v)

for each neighbor u in reverse(neighbors):

if u is not visited:

mark u as visited

push u onto S

Comparison Table:

Feature	Recursive DFS	Iterative DFS
Implementation	Uses system call stack	Uses explicit stack (array/list)
Code readability	Cleaner, shorter, easier	Longer, more boilerplate
Memory handling	Stack frames auto-managed	Manual stack handling
Large graph safety	Risk of stack overflow	Safer (no recursion limit)
Control	Less flexible	More control over traversal

Dijkstra Algorithm

What problems can arise when using Dijkstra's algorithm in the presence of 1) self-loops, 2) negative weights, and 3) negative cycles?

Case 1. Self-loops:

A **self-loop** is an edge from a node to **itself**, like $A \rightarrow A$.

Problem in Dijkstra-

If the weight is **positive**, it doesn't affect shortest paths, because Dijkstra will always ignore it (the distance from a node to itself is already 0).

If the weight is **negative**, it can wrongly decrease the distance to the same node ($d[u]$), which is meaningless.

Solution-

If such a loop exists, treat it as an **invalid graph input** or Before running Dijkstra, **remove all self-loops** because they don't contribute to shortest paths.

Case 2. Negative weights(but no negative cycles)

An edge with a **negative weight**, like $A \rightarrow B$ with weight -3.

Problem in Dijkstra-

Dijkstra fails here.

- Dijkstra **assumes** that once a node is visited (its shortest distance is found), it won't be updated again.
- With **negative weights**, a better path might be found **after** a node has already been "finalized", breaking this assumption

Solution-

For single-source shortest paths, **Bellman-Ford algorithm will work. This** Works with negative edges and Runs in $O(V \cdot E)$ time.

Single Source Shortest Path

Bellman - Ford algorithm

(follows dynamic programming strategy)

Steps:

- Repeatedly Relax all edges for $(n-1)H$
- $n = \text{no. of vertices}$

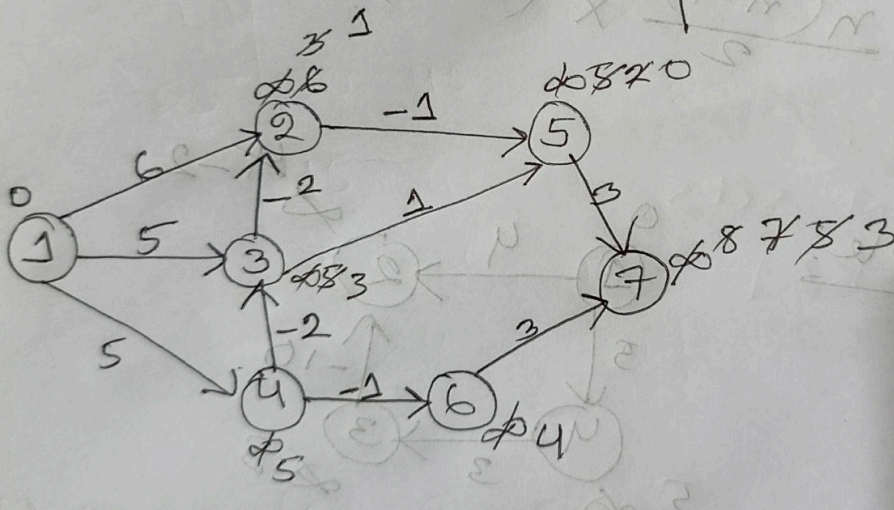
$$|V| = n = 7$$

$$\text{Relax} = |V| - 1 = 6$$

Relaxation

$$\text{if } (d[u] + c(u,v) < d[v])$$

$$d[v] = d[u] + c(u,v)$$



1	0
2	1
3	3
4	5
5	0
6	4
7	3

EdgeList $\rightarrow (1,2), (1,3), (1,4), (2,5), (3,2), (3,5), (4,3), (4,6), (5,7), (6,7)$

Note: we can stop when $d[v]$ stop changing

Case 3. Negative cycle:

Problem in Dijkstra-

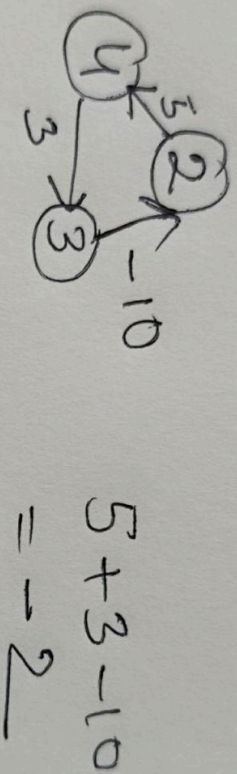
Dijkstra fails here. And even Bellman-Ford fails here. But Bellman-Ford can detect negative cycles. After $|V|-1$ relaxations, if any distance still decreases, a negative cycle exists.

If there's a cycle whose total weight is negative, we will keep looping around it and reduce the path cost indefinitely.

Solution-

Report that "shortest path does not exist".

Draw Back of Bellman-Ford:
This will fail to perform if there is a cycle and the total weight of cycle is negative:



Bellman-Ford algorithm can detect this. After $|V|-1$ relaxations, we can perform relaxation one more time and check, if any distance change, it means a negative weight cycle exist.

Pseudocode for Dijkstra

```
function Dijkstra(Graph, source):  
    dist[v] =  $\infty$  for all v  
    dist[source] = 0  
    PQ = priority queue with (0, source)  
  
    while PQ not empty:  
        (d, u) = PQ.extract_min()  
        if d > dist[u]: continue  
  
        for each edge (u, v, w) in Graph:  
            if dist[u] + w < dist[v]:  
                dist[v] = dist[u] + w  
                PQ.insert(dist[v], v)  
  
    return dist[]
```

#HW done by Tahmid#

1/0 knapsack if not sorted

When the **profits** and **weights** are not sorted, we can **still solve the 0/1 Knapsack problem** using the same **dynamic programming (DP)** approach. The process remains the same, but we need to ensure that we consider the **profits and weights** together and not treat them independently.

Approach:

1. **Do not sort** the profits or weights. Treat them as pairs of **(profit, weight)**.
2. **Apply the standard 0/1 Knapsack DP** approach:
 - Let $dp[i][w]$ represent the maximum profit achievable with the first i items and capacity w .
 - Recurrence:
 - If the weight of the current item exceeds the capacity w , we cannot include it: $dp[i][w] = dp[i-1][w]$.
 - If the weight of the current item is less than or equal to w , we have two choices:
 - Exclude the item: $dp[i][w] = dp[i-1][w]$
 - Include the item: $dp[i][w] = dp[i-1][w - \text{weight}[i]] + \text{profit}[i]$
 - So, $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{profit}[i])$.

Example:

Given the **profits** and **weights**:

- **Profits:** {2, 5, 3, 6}
- **Weights:** {4, 3, 1, 2}
- **Capacity:** 5

1. Pair profits and weights

We create pairs of profits and weights:

[(2, 4), (5, 3), (3, 1), (6, 2)]

2. Initialize the DP table

The **DP table** is a 2D table where $dp[i][w]$ represents the maximum profit we can get with the first i items and a total capacity of w .

DP Table Dimensions:

- Rows = number of items (4 in this case)
- Columns = the capacity of the knapsack (0 to 5, i.e., 6 columns)

Let's now build the table and solve it.

I'll calculate the values for this case using the DP method and provide the resulting table.

Here's the solution with the **unsorted** profits and weights:

Given:

- **Profits:** {2, 5, 3, 6}
- **Weights:** {4, 3, 1, 2}
- **Capacity:** 5

1. DP Table:

i\w	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (4, 2)	0	0	0	0	2	2

2 (3, 5) 0 0 0 5 5 5

3 (1, 3) 0 3 3 5 8 8

4 (2, 6) 0 3 6 9 9 **11**

2. Best Profit: 11

3. Chosen Items: Item 4 (weight 2, profit 6) and Item 2 (weight 3, profit 5)

- Total weight = $2 + 3 = 5$
- Total profit = $6 + 5 = 11$

This gives the **maximum profit of 11** with a total weight exactly filling the capacity of 5.

Backtracking:

- **Item 4** (weight 2, profit 6) is included (capacity reduced from 5 to 3).
- **Item 2** (weight 3, profit 5) is included (capacity reduced from 3 to 0).

Coin example of knapsack

Let's break down the knapsack problem step-by-step, covering all the variants related to coins. I'll guide you through the theory, the approach, and the solution for each of them, so by the end, you'll be comfortable solving any knapsack problem that involves coins.

Key Variants of the Knapsack Problem with Coins:

1. Unbounded Knapsack (Coin Change)
 2. Bounded Knapsack with Coins
 3. 0/1 Knapsack with Coins
 4. Multi-dimensional Knapsack with Coins
 5. Minimum Coin Change Problem
 6. Multi-objective Knapsack with Coins
-

1. Unbounded Knapsack (Coin Change)

Problem Setup:

- You have unlimited coins for each denomination.
- The goal is to find the maximum value you can achieve by filling the knapsack of capacity W using the coins.
- In the coin change problem, you typically focus on minimizing the number of coins needed to form a target amount.

Example:

- Coins: {1, 3, 5}
- Target Amount: 8

Approach:

- We will use dynamic programming (DP) to solve this. The idea is to iteratively calculate the minimum coins required for each amount from 0 to 8.

DP Formula:

- $dp[i]$: Minimum number of coins required to make the amount i .
- Recurrence:
 - $dp[i] = \min(dp[i], dp[i - \text{coin}] + 1)$ for each coin in available coins (only if $i \geq \text{coin}$).

DP Table Construction:

We initialize $dp[0] = 0$ (since no coins are needed to make amount 0) and all other values to infinity (float('inf')) initially.

2. Bounded Knapsack with Coins

Problem Setup:

- You have a limited number of coins of each denomination.
- The goal is to maximize the total value without exceeding the knapsack's capacity, and each coin type has a limited count.

Example:

- Coins: {1, 3, 5}
- Counts: {2, 1, 1}
- Capacity: 8

Approach:

- Use dynamic programming but modify it to account for the limited quantity of each coin type.
- We break each coin into "groups" or use a strategy similar to the bounded knapsack.

- We process each coin, ensuring we don't take more than the allowed number.
-

3. 0/1 Knapsack with Coins

Problem Setup:

- You have a set of coins and must choose one or none from each denomination.
- The goal is to find whether it's possible to form an exact amount with the available coins, or find the maximum value achievable with this restriction.

Example:

- Coins: {1, 3, 5}
- Capacity: 8

Approach:

- This is a 0/1 knapsack problem. Each coin can either be included or excluded from the knapsack.
- Use the DP method to check if we can form the exact amount and also to find the maximum possible value.

Recurrence:

- Similar to bounded knapsack but each coin can be used only once:

- $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{coin}[i]] + \text{value}[i])$

4. Multi-dimensional Knapsack with Coins

Problem Setup:

- You have multiple constraints (e.g., weight and value).

- You want to maximize a certain value while respecting multiple dimensions, such as coin weight and its corresponding value.

Example:

- Coins: {1, 2, 3}
- Values: {5, 3, 7}
- Weights: {2, 3, 4}
- Capacity (weight): 8

Approach:

- We use multi-dimensional dynamic programming where we handle multiple constraints (e.g., weight and value).
 - The DP array will have more dimensions (2D or 3D, depending on the number of constraints).
-

5. Minimum Coin Change Problem

Problem Setup:

- You are given an array of coin denominations and a target amount. The goal is to find the minimum number of coins required to form that amount.

Example:

- Coins: {1, 3, 5}
- Amount: 8

Approach:

- This is a variation of the unbounded knapsack problem where we focus on finding the minimum coins required to form the target value.

- Use dynamic programming to solve it by updating the DP table for each coin denomination.
-

6. Multi-objective Knapsack with Coins

Problem Setup:

- You want to maximize two or more objectives, such as minimizing the number of coins while maximizing value.
- This can be solved using a multi-objective optimization approach.

Example:

- Coins: {1, 3, 5}
- Target: Form 8 units of value while using the fewest coins.

Approach:

- Solve this using both dynamic programming and optimization methods, considering both objectives simultaneously.
-

Detailed Walkthrough for Unbounded Knapsack (Coin Change)

Let's work through an example of the coin change problem to really understand the steps:

Example:

Coins: [1, 3, 5]

Target Amount: 8

1. Initialization:

- Create a DP array **dp[]** of size **target_amount + 1** (9 elements in this case).
- Initialize **dp[0] = 0** (0 coins needed to form 0 amount).

- Set all other values to infinity (**inf**) to represent that the amount is initially unreachable.

2. Recurrence Relation:

- For each coin, we update the DP array:
 - $dp[amount] = \min(dp[amount], dp[amount - coin] + 1)$ for all $amount \geq coin$.

3. Result:

- The value at $dp[target_amount]$ will give us the minimum number of coins required.

Example Solution Code:

Let's go ahead and write a solution for the coin change (unbounded knapsack) problem using dynamic programming.

For the coin change problem with coins = [1, 3, 5] and target amount = 8, the minimum number of coins required to make the amount is 2.

Explanation:

- The minimum number of coins required to form 8 is obtained by using one coin of 3 and one coin of 5, i.e., $3 + 5 = 8$.
- Hence, the solution is 2 coins.

Let's dive deep into 0/1 knapsack with coins, step-by-step. This is a classic knapsack problem, where you are given a set of items (coins), each with a weight (coin value) and profit (coin value), and you can either include or exclude each item (coin) from your knapsack. The goal is to maximize the total profit without exceeding the knapsack's capacity.

0/1 Knapsack with Coins: Problem Setup

Problem Definition:

1. **Items:** A set of coins, each with:
 - **Weight:** The weight (value) of the coin.
 - **Profit:** The profit (value) of the coin.
 2. **Capacity:** The maximum weight (capacity) that the knapsack can hold.
 3. **Goal:** Maximize the total profit of the coins you select, such that the total weight does not exceed the knapsack's capacity. Each coin can be included at most once.
-

Example Problem

- **Coins (items):** {1, 3, 5}
- **Weights:** {1, 3, 4}
- **Profit:** {2, 4, 5}
- **Capacity:** 5

Steps for Solving the 0/1 Knapsack with Coins Problem

Step 1: Understand the DP Table

We will use a dynamic programming (DP) table $dp[i][w]$ to store the maximum profit that can be obtained with a knapsack capacity w using the first i coins.

Let $dp[i][w]$ represent the maximum profit we can get with the first i coins and a knapsack of capacity w .

- If we don't include the current coin, the profit remains the same as $dp[i-1][w]$.
- If we do include the current coin, the profit will be $profit[i] + dp[i-1][w - weight[i]]$.

Recurrence Relation:

- $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - weight[i]] + profit[i])$

Where:

- $dp[i-1][w]$: Profit without including the current item.
- $dp[i-1][w - \text{weight}[i]] + \text{profit}[i]$: Profit by including the current item.

Step 2: Base Case

- $dp[0][w] = 0$: No items lead to a profit of 0 for any capacity.
- $dp[i][0] = 0$: With capacity 0, no items can be included, so the profit is 0.

Step 3: Build the DP Table

The table will be filled from $dp[1][1]$ to $dp[n][W]$, where n is the number of items (coins) and W is the knapsack's capacity.

Solution Approach:

1. Initialization:

- Create a DP table of size $(n+1) \times (W+1)$ where n is the number of coins and W is the knapsack capacity.
- Initialize the first row and first column to 0 (base case).

2. Fill the Table:

- For each item (coin) and each possible capacity, use the recurrence relation to compute the maximum profit.
-

Let's Solve the Example Step-by-Step

- Coins: {1, 3, 5}
- Weights: {1, 3, 4}
- Profits: {2, 4, 5}
- Capacity: 5

We will now calculate the DP table and track the optimal solution.

Solution for the 0/1 Knapsack with Coins Problem

Given the following inputs:

- Coins: {1, 3, 5}
- Weights: {1, 3, 4}
- Profits: {2, 4, 5}
- Capacity: 5

DP Table:

This table shows the maximum profit achievable for each combination of the first i coins and knapsack capacities from 0 to 5.

i\w	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (1, 2)	0	2	2	2	2	2
2 (3, 4)	0	2	2	4	6	6
3 (4, 5)	0	2	2	4	6	7

Best Profit: 7

- Selected Items: Item 3 (index 2, weight 4, profit 5) and Item 1 (index 0, weight 1, profit 2).
- Total Profit = 5 (from Item 3) + 2 (from Item 1) = 7.

- **Total Weight = 4 (from Item 3) + 1 (from Item 1) = 5, which is exactly the knapsack capacity.**

Explanation:

- 1. Item 1 is included (weight 1, profit 2).**
- 2. Item 3 is included (weight 4, profit 5).**
- 3. These selections give a total profit of 7, which is the maximum value that can be achieved without exceeding the capacity.**

Kruskal's Algorithm

How to do Kruskal's algorithm implementation using a disjoint set? [Farhan]

Kruskal's algorithm is used to find a **Minimum Spanning Tree (MST)** of a graph. MST is a subset of edges that connects all vertices with:

1. No cycles
2. Minimum total weight

Step-by-step idea:

1. Sort all edges by weight (smallest first).
2. Initialize **Disjoint Set Union (DSU)** where each node is its own parent.
3. Pick edges one by one (smallest first).

If the edge connects two **different sets** → include it in MST.

If the edge connects nodes in the **same set** → skip (it would make a cycle).

4. Repeat until you add **(V-1) edges** (V = number of vertices).

We need **DSU** to **detect cycles** efficiently. In **DSU**:

find(x): returns the representative/root (boss) of the set that x belongs to.

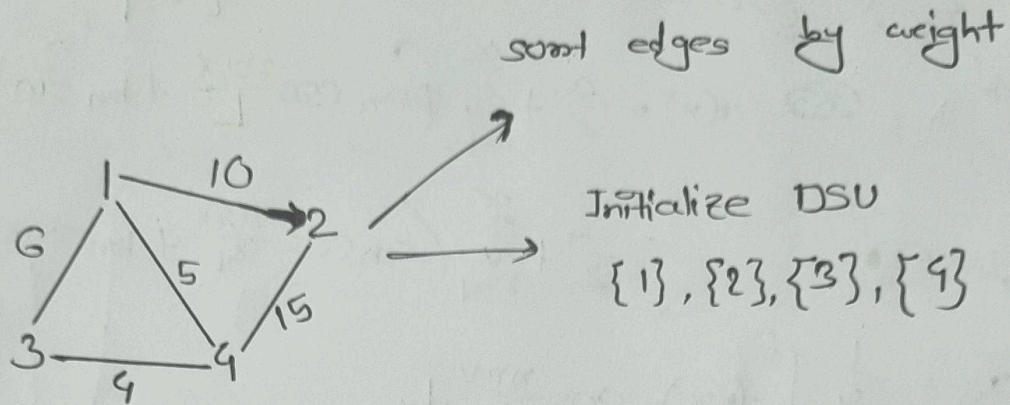
union (x, y): merges the sets of x and y.

Optimizations:

Path compression (make future find faster).

Union by rank/size (attach smaller tree under larger tree)

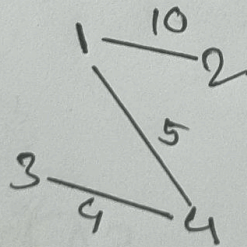
SIMULATION



Process edge one by one,

- ① $(3-4, 4) \rightarrow$ merge (3) and (4). cost = 4
- ② $(1-4, 5) \rightarrow$ merge (1) with (3,4). cost = 9
- ③ $(1-3, 6) \rightarrow$ 1, 3 are in same set. (skip)
- ④ $(1-2, 10) \rightarrow$ merge (2) with (1,3,4). cost = 19
- ⑤ Done (3 edges taken = $N-1=3$)

Final MST:



Pseudocode

read N, M // N = number of nodes, M = number of edges

create edgeList = empty list

for $i = 1$ to M :

 read u, v, w

 add (u, v, w) to edgeList

// Step 1: Sort edges by weight

sort edgeList by w (ascending)

// Step 2: Initialize DSU

parent[i] = i for all i from 1 to N

rank[i] = 1 for all i from 1 to N

// Step 3: Define Find function

function find(x):

 if parent[x] $\neq x$:

 parent[x] = find(parent[x]) // path compression

 return parent[x]

// Step 4: Define Union function

function union(x, y):

 rootX = find(x)

 rootY = find(y)

```

if rootX == rootY:
    return false // same set → cycle

if rank[rootX] < rank[rootY]:
    parent[rootX] = rootY
else if rank[rootX] > rank[rootY]:
    parent[rootY] = rootX
else:
    parent[rootY] = rootX
    rank[rootX] = rank[rootX] + 1
return true

```

// Step 5: Kruskal's Algorithm

```

mstCost = 0
mstEdges = empty list
for each (u, v, w) in edgeList:
    if union(u, v) == true:
        mstCost = mstCost + w
        add (u, v, w) to mstEdges

```

// Step 6: Print result

```

print "MST Cost = ", mstCost
for each (u, v, w) in mstEdges:
    print u, "-", v, ":", w

```